Diario de desarrollo

2.0) Diseño máquinas de estado

Comportamiento de fantasmas

He empezado pensando en las distintas acciones, dividiendolas en acciones de ataque, defensivas, preventivas y “encerrado”. Estas son las acciones que pensé preliminarmente:

- **Ataque**: Perseguir, interceptar, bloquear power pill.
- **Defensa**: Huir, refugiarse en compañero, dispersarse.
- Encerrado

Después le pregunté a la IA Claude como afinar esas acciones preliminares:

Prompt

Mis acciones actuales para una máquina de estados de los fantasmas son las siguientes, divididas en 3 categorías: Ataque: Perseguir (ir a la posicion de mspacman), Interceptar (intentar flanquear a mspacman si ya hay otro compañero persiguiendo), Bloquear power pill (Si se puede llegar a power pill antes que mspacman va hacia ella para bloquearla). Defensa: Huir (alejarse de mspacman, evitando ir todos por el mismo camino), refugiarse en compañero (si soy comestible y un compañero fantasma no, iré hacia él para que me proteja), dispersarse (si mspacman está lejos, ir a distintas esquinas para evitar ser comidos varios a la vez si se acerca). Encerrado (he sido comido y tengo que esperar a que vuelva mi turno). Dime como podriamos afinar estos comportamientos, cuales podríamos añadir?

Respuesta

Tienes una buena base: ya cubre perseguir, cortar el paso, defender la power pill y coordinarse cuando hay comestibles. Te propongo afinar cuatro de tus acciones y añadir tres nuevas.

Cómo afinar las que tienes

Perseguir. Ir a la posición de MsPacMan por el camino más corto, midiendo siempre la distancia con el último movimiento del fantasma, porque no puede darse la vuelta. Conviene que solo persiga **uno**: el más cercano a MsPacMan. Cuatro fantasmas persiguiendo por el camino más corto acaban en fila por el mismo pasillo, y en fila no cierran ningún bucle.

Interceptar. En lugar de ir a la posición de MsPacMan, ir a su **próximo cruce**, siguiendo el pasillo por el que va hasta el siguiente cruce. Perseguir es ir siempre por detrás;

interceptar es esperarla donde va a llegar. Así el perseguidor la empuja y el interceptor le cierra la salida.

Bloquear power pill. La clave es la condición de "llegar antes". Compara la distancia del fantasma **a la pill** con la de MsPacMan **a la misma pill**, con un margen generoso: llegar a la vez es lo peor que puede pasar, porque MsPacMan se la come con el fantasma al lado. Basta con que la bloquee uno, el que antes llegue.

Huir → huida eficiente. Dos mejoras:

- **Elegir bien la salida.** Huir con `getNextMoveAwayFromTarget` hace que casi todas las salidas empaten, y el empate se resuelve por el orden del `enum MOVE`, así que todos los fantasmas huyen por el mismo lado. Mejor descartar el pasillo por el que viene MsPacMan y elegir entre los demás, evitando los que usan otros comestibles.
- **Huir antes, no después.** Si MsPacMan va a llegar a una power pill antes que el fantasma (justo el caso contrario a bloquear), lo mejor es empezar a huir **ya**, alejándose de la pill. Huir cuando ya eres comestible es tarde.

Qué añadir

Categoría	Acción	Qué hace
Defensa	**Volver al ataque**	Si el tiempo de comestible que le queda es menor que `2d/3` (siendo `d` la distancia a MsPacMan), deja de huir y vuelve a perseguir. Yendo hacia ella se acercan 3/2 nodos por tick, así que cuando se encuentren ya no será comestible. Gana posición mientras los demás siguen huyendo.
Estándar	**Vigilar power pill**	Situarse cerca de la power pill activa más cercana a MsPacMan **antes** de que ella vaya hacia allí. Es la versión preventiva de bloquear: el fantasma que vigila es el mejor candidato para bloquear cuando llegue el momento.
Estándar	**Patrullar píldoras**	Moverse hacia la zona del laberinto donde quedan más píldoras. MsPacMan tiene que ir allí tarde o temprano para terminar el nivel, así que el fantasma la espera donde va a tener que pasar, en lugar de vagar.

Con estas, tu lista quedaría:

Ataque	Defensa	Estándar
Perseguir	Huida eficiente	Vigilar power pill
Interceptar	Refugiarse en compañero	Patrullar píldoras
Bloquear power pill	Dispersarse	Encerrado
	Volver al ataque	

Para que funcionen juntas

Lo más importante es que **los cuatro fantasmas no hagan lo mismo a la vez**. Una forma sencilla, sin necesidad de mensajes, es repartir los papeles cada tick según la distancia a MsPacMan: el más cercano persigue, el segundo intercepta, el tercero vigila la power pill y el cuarto patrulla. Como los cuatro ven el mismo estado, todos llegan al mismo reparto. Y como se recalcula cada tick, el papel de perseguidor es del que esté más cerca en cada momento, no siempre del mismo fantasma.

Después diseñé la máquina de estados, con 4 superestados (normal, comestible, peligro por PP y encerrado), con sus transiciones. Para el superestado de “normal”, pedí ayuda a Claude para completar las transiciones, adjuntando el estado sin transiciones:

-

Prompt

Esta es mi máquina de estados, ahora hagamos superestado de "normal" por dentro, debe de empezar en perseguir, y pasar a interceptar si ya hay algun compañero persiguiendo, cual podría ser la condicion de entrada vigilar (quiero que vaya a la power pill mas cercana a mspacman para que sea posible bloquear la pildora de poder si es necesario)? Y de patrullar pildoras? Quiero que los fantasmas sean agresivos y no estén esperando a que pacman vaya a por sus pildoras.

-

Respuesta

La idea es repartir los cuatro papeles **por prioridad**: perseguir, interceptar, vigilar y patrullar. Cada fantasma ocupa el papel más importante que quede libre, y si uno superior se queda vacío (porque su fantasma se vuelve comestible o pasa a PELIGRO), sube el mejor candidato.

Transiciones dentro de NORMAL

Estado inicial: **PERSEGUIR**.

Desde	Hacia	Condición
PERSEGUIR	INTERCEPTAR	Ya hay un compañero persiguiendo (*)
INTERCEPTAR	VIGILAR	Quedan power pills, nadie vigila y es el más cercano a la pill objetivo (**)
INTERCEPTAR	PATRULLAR	Ya hay un compañero interceptando y otro vigilando (***)
Cualquiera	PERSEGUIR	Nadie persigue y es el más cercano a Ms. Pacman
VIGILAR, PATRULLAR	INTERCEPTAR	Nadie intercepta y es el más cercano a Ms. Pacman
VIGILAR	INTERCEPTAR	No quedan power pills
PATRULLAR	VIGILAR	Nadie vigila, quedan power pills y es el más cercano a la pill objetivo

(*) Si dos están persiguiendo a la vez, sigue el más cercano a Ms. Pacman y el otro pasa a interceptar. Si empatan, se desempata por el orden del enum `GHOST`.

(**) La pill objetivo es la power pill activa más cercana a Ms. Pacman. Vigila el fantasma (no perseguidor) más cercano a ella, porque será el que antes llegue si hay que bloquearla. Cuando Ms. Pacman se acerque a la pill, pasará a PELIGRO POR PÍLDORA y será el candidato natural a BLOQUEAR.

(***) Para que sea agresivo, el objetivo de patrullar es **la zona de píldoras más cercana a Ms. Pacman**, no la zona con más píldoras. Así el fantasma se coloca por delante de ella, donde va a comer, en lugar de esperar en otra parte del mapa.

Por qué es agresivo

- Los **cuatro papeles se mueven alrededor de Ms. Pacman**: perseguir va a ella, interceptar a su próximo cruce, vigilar a la pill más cercana a ella y patrullar a la comida que tiene más cerca.
- Ningún fantasma se queda esperando en una zona lejana.
- Si el perseguidor desaparece, el más cercano ocupa su puesto enseguida, así que siempre hay alguien encima de ella.

Después terminé de completar el diagrama.

Comportamiento de Ms Pacman

El comportamiento de MsPacman es tal cual siempre ha sido, evitar los fantasmas mientras está movilizándose por los pasillos comiendo píldoras para acumular puntos y luego poder comer la píldora de súper poder que otorga una franja de tiempo corto para poder comer los fantasmas y ser inmune a ser cazados por ellos.

Acciones

Estas son las capacidades físicas y decisiones de movimiento que Pac-Man puede ejecutar en el mapa. Cada una representa una estrategia de navegación activa:

- **Comer píldora:** Ir hacia la píldora común activa más cercana del mapa utilizando la ruta más corta posible.
- **Ir a la píldora de poder:** Dirigirse hacia la píldora de poder (`PowerPill`) activa más cercana calculando la ruta euclidiana óptima.
- **Huir:** Desplazarse en dirección opuesta al fantasma no comestible más cercano para maximizar la distancia de escape.
- **Comer fantasma:** Perseguir al fantasma comestible (`isGhostEdible`) más cercano utilizando la ruta de aproximación más rápida.
- **Evitar camino:** Cambiar la trayectoria actual hacia un nodo alternativo cuando el camino directo hacia una píldora está interceptado por un fantasma.

Las Transiciones

Convertimos las transiciones en variables lógicas y distancias numéricas exactas basadas en nodos (medidas por distancia Euclídea `Constants.DM.EUCLID`).

Para evitar que Pac-Man se quede "titubeando" (entrando y saliendo de un estado repetidamente en un mismo pasillo), aplicamos **histéresis** con un margen de seguridad de **5 nodos**:

De "Comer píldora" a "Huir" (Y viceversa)

- **Condición de Ida (Entrada a Huir):** `nearestGhost != null` **Y** `!game.isGhostEdible(nearestGhost)` **Y** la distancia al fantasma es **menor a 25 nodos**.
- **Condición de Vuelta (Salir de Huir):** La distancia al fantasma peligroso vuelve a ser **mayor o igual a 30 nodos** (Margen de 5 nodos para estabilizar el comportamiento), o el fantasma pasa a ser comestible.

De "Comer píldora" a "Comer fantasma" (Y viceversa)

- **Condición de Ida (Entrada a Cazar):** `nearestGhost != null` **Y** `game.isGhostEdible(nearestGhost)` **Y** la distancia al fantasma comestible es **menor a 40 nodos**.

- **Condición de Vuelta (Salir de Cazar):** El tiempo de vulnerabilidad del fantasma termina (`!isGhostEdible`), el fantasma es devorado, o la distancia al fantasma supera los **45 nodos**.

De "Comer píldora" a "Ir a la píldora" (Y viceversa)

- **Condición de Ida (Peligro preventivo):** `nearestGhost != null` **Y** la distancia entre Pac-Man y la píldora de poder más cercana es **menor o igual al parámetro** `limit` (por ejemplo, 15 nodos), anticipando que necesitará el poder pronto.
- **Condición de Vuelta (Volver al bucle normal):** La píldora de poder es consumida, o la distancia a la píldora de poder vuelve a ser **mayor a** `limit` **+ 5 nodos** si Pac-Man decide cambiar de rumbo.

De "Ir a la píldora" a "Evitar camino"

- **Condición de Ida (Ruta interceptada):** El camino hacia la píldora de poder contiene un fantasma no comestible a una distancia **menor a 12 nodos** del objetivo.
- **Condición de Vuelta (Ruta liberada):** El fantasma se aleja del camino de la píldora a una distancia **mayor a 17 nodos**.

Estados:

- **Huir:** Estado por default, donde aparte de buscar las píldoras para sumar puntos y las píldoras de poder, al momento de que comience en su estado inicial está buscando las píldoras y huyendo de los fantasmas, huyendo por el pasillo donde esté lo más lejos de los fantasmas, luego de haber comido una píldora de poder y que termine el tiempo de efecto de esta, vuelve al estado de huir.
- **Ir a la píldora:** Estamos en este estado cuando huimos de los fantasmas y estamos obteniendo puntos, el objetivo siempre será obtener la pildora de poder para que MsPacman pueda comerse los fantasmas y obtener más puntos
- **Comer Pildora:** Transisionamos a este estado cuando ingerimos la pildora, estado donde aumentamos nuestra velocidad para poder comer los fantasmas en una franja de tiempo, luego del efecto volvemos al estado huir, evitando contacto con los fantasmas.
- **Evitar camino:** Un estado implicito del fantasma donde siempre que está corriendo la cuadrilla, analiza que camino es mejor para tomar el que menos fantasma tenga o el camino que menos nos dirija a uno donde haya fantasmas.

Estado actual	Transición	Método
Default	nearestGhost == null	TowardsTarget(ActivePills)
Comer píldora	nearestGhost != null && !isGhostEdible	AwayFromTarget(Ghost)

Comer píldora	nearestGhost != null && isGhostEdible	TowardsTarget(Ghost)
Ir a la pildora	Fantasma bloquea ruta	TowardsPill

2.1) Codificación de máquinas de estado (P2)

2.1.1 Comportamiento de fantasmas

Una vez con el diseño hecho, para implementar el comportamiento, empiezo creando la estructura de los paquetes.
Ya que todas mis acciones van a compartir unos comportamientos, como “ir hacia un sitio”, entonces le pedí a Claude que elaborase una clase “Moviemientos” dentro del paquete Utils para reunir esas acciones que comparten muchos comportamientos.

Prompt

Crea una clase de utilidades “Movimientos”, con métodos que compartan las acciones para no repetir código. Mide siempre las distancias por el laberinto y con el último movimiento, porque nadie puede darse la vuelta. Por ejemplo: - “irHacia(game, ghost, destino)”: la salida legal que más acerca al fantasma a un nodo. - “huidaEficiente(game, ghost, peligro, ultimoPeligro)”: huir descartando el pasillo por el que viene el peligro y evitando ir en fila con otros comestibles. - `proximoCrucePacman(game)`: el cruce al que llegará Ms. Pac-Man, para interceptar... Explica de forma breve qué hace cada método

---

Respuesta

Clase practica2.grupoHEC.ghosts.utils.Movimientos

Después implementé la clase GhostsInput, la responsable de recoger toda la información sobre el juego para que los fantasmas tomen decisiones.

Prompt

Crea la clase GhostsInput (extends Input), usando GhostsConstants y Movimientos. La leen las transiciones de las cuatro máquinas, así que debe calcular una vez por turno todo lo que necesitan, para que cada transición quede en una línea.
Debe recoger:
- De Ms. Pac-Man: su nodo, su último movimiento, la power pill activa más cercana a ella y su distancia a esa pill.
- De cada fantasma: si está en la cárcel, si es comestible, su tiempo comestible, y su distancia a Ms. Pac-Man y a la pill (siempre con su último movimiento).

Y ofrecer las condiciones del diagrama:
- (*I) cerca de la pill amenazada, y su salida con histéresis (Pacman se aleja de la PP).
- (*II) fuera de peligro: tiempoComestible + MARGEN_HUIR < 2d/3.

- (*III) llega antes a la PP, y una versión con más ventaja para pasar de HUIDA a BLOQUEAR, de forma que como mucho un fantasma empiece a bloquear cada turno.
- En COMESTIBLE: Ms. Pac-Man cerca / lejos (con histéresis) y si puede refugiarse en un compañero no comestible al que llega antes de que Ms. Pac-Man lo alcance.
- En NORMAL, el reparto de papeles por prioridad (nota *V): perseguidor el más cercano a Ms. Pac-Man, luego interceptor, luego vigilante (el más cercano a la PP, si queda alguna) y el resto patrulla. Si el perseguidor deja de estar en NORMAL, otro ocupa su lugar en el mismo turno, y añade histéresis para que los papeles no cambien en cada cruce.
- Para los comestibles que se dispersan, un reparto de esquinas: una distinta para cada uno.

Explica brevemente cada parte.

Respuesta

He creado `GhostsInput.java` y una clase nueva, `Reparto.java`, y he actualizado `GhostsConstants.java` y `Movimientos.java`. Las cuatro compilan juntas, pero lo he comprobado contra versiones simplificadas de `Game` e `Input`, no contra el motor real, y no se han ejecutado en una partida.

Por qué hace falta `Reparto`

Lo que pedías obliga a recordar el turno anterior, y `GhostsInput` se crea nuevo en cada turno:

- **Histéresis en los papeles**: para que el titular conserve su papel, hay que saber quién lo tenía.
- **Esquinas estables**: un comestible no debe cambiar de esquina a mitad de camino.
- **Las acciones necesitan el reparto**: `DispersarseAction` tiene que saber su esquina, pero las acciones solo reciben el `Game`, no el `Input`.

`Reparto` guarda los papeles, las esquinas y los refugios de un turno. El controlador conserva el último y se lo pasa al `GhostsInput` del siguiente: `new GhostsInput(game, repartoAnterior)`. Las acciones pueden recibir una referencia al reparto actual. Recordar dentro de la partida está permitido; lo prohibido es guardar datos entre partidas.

Reparto de papeles en NORMAL

Se calcula entre los fantasmas que están en NORMAL según el juego: fuera de la cárcel, no comestibles (o fuera de peligro) y sin una pill amenazada cerca. Los papeles se asignan **por prioridad**, cada uno al mejor de los que quedan libres:

1. **PERSEGUIR**: el más cercano a Ms. Pac-Man.
2. **INTERCEPTAR**: el siguiente más cercano a Ms. Pac-Man.
3. **VIGILAR**: de los que quedan, el más cercano a la power pill. Solo si queda alguna.

4. **PATRULLAR**: el resto.

Si el perseguidor desaparece, porque se vuelve comestible o pasa a PELIGRO, deja de ser candidato. El papel queda libre y en ese mismo turno lo ocupa el más cercano de los que quedan; los papeles inferiores se recolocan detrás.

La histéresis (`HISTERESIS_PAPELES = 3`, nueva en `GhostsConstants`): si el titular sigue en NORMAL, solo pierde el papel frente a otro que esté **al menos 3 nodos más cerca**. Así dos fantasmas casi a la misma distancia no se intercambian el papel en cada cruce. Los empates se resuelven por el orden del enum `GHOST`, para que el resultado sea siempre el mismo.

Reparto de esquinas en COMESTIBLE

Cada comestible recibe una esquina distinta:

1. **Quien ya tenía esquina la conserva**, si sigue libre.
2. Las que faltan se asignan de una en una, eligiendo la pareja (fantasma, esquina) con **más margen**: lo que tarda Ms. Pac-Man en llegar a esa esquina menos lo que tarda el fantasma, contando que va a media velocidad. Así ningún fantasma huye hacia una esquina que Ms. Pac-Man tiene más cerca.

Para esto he añadido `Movimientos.esquinas(game)`, que devuelve los cuatro cruces de esquina, y he quitado `esquinaDe`, que asignaba la esquina por el orden del enum.

Dos detalles de implementación

- **Orden de construcción.** `super(game)` llama a `parseInput()`, que calcula las distancias y condiciones de cada fantasma. El reparto se calcula después, en el cuerpo del constructor, porque necesita el reparto anterior. Por eso ningún atributo tiene valor inicial: se borraría lo que acaba de calcular `parseInput()`.
- **Sin redondeos.** Las condiciones con `2d/3` están multiplicadas por 3 a ambos lados (por ejemplo, `3·(t + MARGEN) < 2·d`) para no perder precisión con la división entera.

El siguiente paso es el esqueleto del paso 3: `EncerradoAction`, `PerseguirAction`, sus transiciones y el controlador `Ghosts`, para ver la primera partida funcionando.

Ahora ya he terminado de implementar un controlador muy basico que solo tiene 2 estados: Encerrado y Perseguir. Para probar el funcionamiento y para ver lo mal que funciona xd. Las puntuaciones han sido:

- MsPacManIntermedio: 2.600 puntos
- MsPacManAvanzado: 22.000 puntos

Tras muchos prompts de IA, en el que superestado a superestado he ido creando las acciones y transiciones, ya tengo una primera version de mi maquina completa! Los resultados han sido los siguientes tras 200 trials:

- **Puntuacion media**: 5673,35
- **Desviación típica**: 2506,69
- **Mínimo / Máximo**: 1170 / 15450
- **Ticks medios**: 2492,73
- **Tiempo total**: 7229 ms

Ahora lo que toca es detectar problemas y poco a poco ir afinando las constantes de histeresis, radios de peligro y PP…

Después de hacer de entrenador de los fantasmas un rato, observando sus partidas, he sacado estos fallos iniciales:

- Hay que aumentar el radio de peligro de las power pill, se acercan demasiado y ya reculan cuando es imposible huir.
- No bloquean PP cuando deberían → quizas hay que quitar un margen para que pueda llegar siempre que pueda.
- El fantasma que intercepta a veces sigue el mismo camino que el que persigue, interesa que flanquee más.

Prompt

Ya tenemos la versión inicial de los fantasmas lista; saca unos 5700 puntos de media (200 partidas). Después de observar varias partidas he detectado tres fallos. Arréglalos cambiando constantes o clases, explicando la causa de cada uno, sin inventar técnicas nuevas y manteniendo los nombres actuales de las clases:

- PELIGRO POR PÍLDORA se activa tarde: los fantasmas empiezan a huir cuando ya están demasiado cerca de la PP. Revisa RADIO_PILL (cuándo se considera amenazada la PP) y RADIO_PELIGRO (qué fantasmas entran en PELIGRO) para que huyan con más ventaja, teniendo en cuenta que al volverse comestibles van a media velocidad.
- No bloquean la PP aunque podrían hacerlo. Este es el más importante. Comprueba: - si el margen exigido para entrar en BLOQUEAR (MARGEN_PP + la histéresis) es demasiado grande. - si el fantasma abandona el bloqueo justo al llegar: no puede pararse sobre la PP, la pasa de largo y la distancia "sin darse la vuelta" hace que (*III) deje de cumplirse; - qué debe hacer BloquearPPAction una vez que está junto a la PP (en vez de dar vueltas y alejarse).
- Al interceptar, el fantasma va muchas veces por el mismo camino que el que persigue. Tiene que franquear: elegir un camino hacia el cruce al que se dirige Ms. Pac-Man que no pase por ella ni por el camino del perseguidor, aunque sea más largo, y usar el más corto solo si no hay otro. Dime qué constantes y clases cambias, y cómo medir el efecto de cada cambio por separado.

Respuesta

He hecho los tres cambios. Compilan contra mi versión simplificada del motor, pero no los he probado en el real. Te explico la causa de cada problema, porque en el bloqueo había algo más que el margen.

1. Radio de peligro (`GhostsConstants`)

Ya no salen a huir cuando están encima de la PP:

- `RADIO_PILL`: 20 → **30**. La PP se considera amenazada antes, con Ms. Pac-Man más lejos.
- `RADIO_PELIGRO`: 30 → **45**. Entran en PELIGRO fantasmas que están más lejos de la PP, así que empiezan a huir con más ventaja. Lo necesitan porque, en cuanto son comestibles, van a media velocidad.

2. No bloqueaban: había tres causas

1. **Margen excesivo.** Para empezar a bloquear se pedían `MARGEN_PP + HISTERESIS_PP` = 7 nodos de ventaja, y con la amenaza a 20 nodos solo podía bloquear un fantasma a 12 nodos o menos de la PP. Ahora `MARGEN_PP = 1` (basta con no empatar) y hay una constante propia, `HISTERESIS_BLOQUEO = 2`, así que se piden 3 nodos de ventaja. `HISTERESIS_PP` queda solo para salir de PELIGRO.
2. **Al conseguirlo, abandonaba.** El fantasma llegaba a la PP, no podía pararse y la pasaba de largo. La distancia “sin darse la vuelta” de vuelta a la PP pasaba a ser enorme, (*III) dejaba de cumplirse y volvía a HUIDA justo cuando ya la estaba bloqueando. Ahora `GhostsInput.distBloqueo` usa la distancia real si el fantasma está a `RADIO_GUARDIA_PP = 4` nodos o menos de la PP: ya la está guardando.
3. **Qué hace al llegar.** `BloquearPPAction` sigue yendo hacia la PP mientras está lejos. Cuando ya la guarda, sale al encuentro de Ms. Pac-Man por el camino por el que viene: o se la come, o la obliga a darse la vuelta. Antes daba vueltas alrededor de la PP y se iba alejando.

3. Interceptar flanqueando

He añadido `Movimientos.irHaciaFlanqueando(game, ghost, destino, evitar)`. Entre las salidas posibles, elige la que llega al destino sin pasar por ningún nodo prohibido, aunque el camino sea más largo, y si no hay ninguna, usa la más corta. `InterceptarAction` prohíbe Ms. Pac-Man y todo el camino del perseguidor hasta ella. Así uno va por detrás y el otro por delante, no en fila.

`InterceptarAction` ahora recibe el `Supplier<Reparto>` para saber quién persigue, y `Ghosts.java` ya se lo pasa.

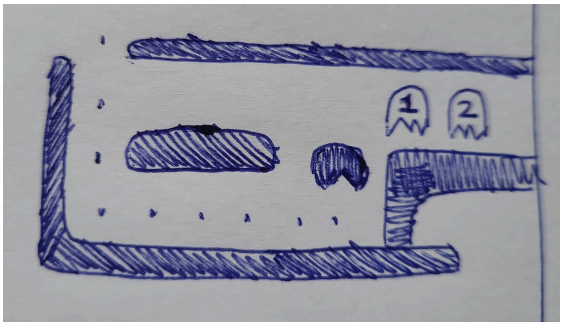
Son tres cambios a la vez. Si la media mejora, no sabremos cuánto aporta cada uno, y si empeora, no sabremos cuál ha sido. Te recomiendo medir las 200 partidas al menos una vez con todo y, si el resultado es raro, deshacer un cambio cada vez.

Estos cambios han mejorado la puntuación. Con **MsPacManAvanzado** ha pasado de **5700** a **5075** de media con 600 trials.

Prompt

Tras varias partidas aun veo estos problemas:

- Cuando pacman se come una PP, se invierte el sentido de todos los fantasmas: los que huían de la PP acaban yendo hacia ella. Al huir en el superestado PELIGRO, elige la salida teniendo en cuenta esa inversión.
- El rol de patrullar píldoras no sirve de mucho: es mejor que ese fantasma también vaya a cazar a pacman, por un camino distinto al del perseguidor y el interceptor.
- Algunos fantasmas comestibles se sacrifican sin intentar huir. Revisa cómo se mide su distancia a pacman.
- Los comestibles no siempre huyen por el mejor camino: prioriza las salidas seguras antes que los pasillos sin compañeros, quizas hay que mirar 2 cruces por delante y no solo uno.
- Los papeles de perseguir e interceptar se intercambian solo porque el interceptor se acerca más, y a veces el más cercano debería interceptar y el otro perseguir para encerrarla. Decide quién intercepta según quién puede adelantarse a ella.
- A veces un fantasma podría atrapar a pacman en su tramo y no lo hace porque su rol es otro, por ejemplo vigilar power pills.



Subí esta foto para explicar el problema de los roles solo por distancias: Si el fantasma 1 persigue y el 2 intercepta, se puede escapar Ms. PacMan, en este caso, el 1 debería interceptar y el 2 perseguir.

Respuesta

Cambios a varias clases, con ello, la puntuación mejoró a **4700** con **MsPacManAvanzado**!

Después, para probar las multiples combinaciones de constantes, como RADIO_PILL, HISTERESIS_PAPELES, DISTANCIA_CERCA_PACMAN… Le he pedido a Claude que me genere una clase que pruebe multiples combinaciones y se quede con la mejor.

Prompt

Crea una clase Afinador (fuera del paquete del grupo) que pruebe valores de las constantes de GhostsConstants una a una, midiendo cada valor con varias partidas contra los tres rivales y quedándose con el que dé menos puntos a Ms. Pac-Man.

Para no elegir por ruido, cambia un valor si mejora al actual por encima de un umbral, que valide al final la configuración ganadora con más partidas y que imprima los valores que mejor resultado han dado.

Prompt

Clase Afinador

Tras 4 minutos de probar constantes y 13.000 partidas, hemos mejorado la puntuación con **MsPacManAvanzado** a **4550**! Una mejora importante.

2.1.2) Implementación del controlador MsPacMan

Necesito un controlador en Java el cual se va a encargar de crear la maquina de estados con la informacion que reciba de Transitions, Actions y MspacmanINput para la Práctica 2 de IA para videojuegos (Ms. Pac-Man). Adjunto: el enunciado, los apuntes del tema (máquinas de estados finitos(FMS), Game.java, Constants.java. y los apuntes de la clase Game.

OBJETIVO
Diseñar e implementar un controlador para Ms. Pac-Man basado en una máquina de estados finitos (FSM), con el propósito de separar las responsabilidades del controlador, organizar los comportamientos del personaje en estados independientes y gestionar los cambios entre ellos mediante transiciones definidas por condiciones del juego. Esta arquitectura busca reducir el acoplamiento entre las distintas responsabilidades, facilitar la comprensión y el mantenimiento del código y permitir incorporar nuevos comportamientos sin concentrar toda la lógica de decisión en una única clase.

Reglas Duras:

- Paquete: es.ucm.fdi.ici.c2627.practica2.grupoXX (mi grupo: ___).
- Sin aprendizaje ni persistencia. Se puede guardar estado dentro de la partida.
- Sin visibilidad parcial, sin mensajes, 40 ms por turno.
- Sin código de depuración (nada de GameView) en ningún fichero.
- Usa solo métodos que existan en el Game.java adjunto. Si necesitas otro, dímelo en vez de suponerlo.
- Nunca restes índices de nodo. Usa DM.PATH y el último movimiento. Comprueba -1 y null donde se reciben. Comprueba la longitud de getShortestPath antes de acceder a camino[1].
- Todos los umbrales y pesos en Constantes, con nombre y un comentario cada uno.

Diseño de clases

- Clase MsPacMan (extends PacmanController) donde se encarga de construir y coordinar la maquina de estados Máximo 500 líneas por comportamiento, los estados son los siguientes:(Huir, comer fantasma, comer pildora comun, ir a la pildora de poder, evitar camino bloqueado,patrullar zona segura).

    - Accion asociada a cada una de los estados:
- Estas son las capacidades físicas y decisiones de movimiento que Pac-Man puede ejecutar en el mapa. Cada una representa una estrategia de navegación activa:

    - **Comer píldora:** Ir hacia la píldora común activa más cercana del mapa utilizando la ruta más corta posible.
    - **Ir a la píldora de poder:** Dirigirse hacia la píldora de poder (PowerPill) activa más cercana calculando la ruta euclidiana óptima.
    - **Huir:** Desplazarse en dirección opuesta al fantasma no comestible más cercano para maximizar la distancia de escape.

    - **Comer fantasma:** Perseguir al fantasma comestible (isGhostEdible) más cercano utilizando la ruta de aproximación más rápida.
    - **Evitar camino:** Cambiar la trayectoria actual hacia un nodo alternativo cuando el camino directo hacia una píldora está interceptado por un fantasma.
- Transiciones que conectan estados:

  Convertimos las transiciones en variables lógicas y distancias numéricas exactas basadas en nodos (medidas por distancia Euclídea Constants.DM.EUCLID).

  Para evitar que Pac-Man se quede "titubeando" (entrando y saliendo de un estado repetidamente en un mismo pasillo), aplicamos **histéresis** con un margen de seguridad de **5 nodos**:

  De "Comer píldora" a "Huir" (Y viceversa)

    - **Condición de Ida (Entrada a Huir):** nearestGhost != null **Y** !game.isGhostEdible(nearestGhost) **Y** la distancia al fantasma es **menor a 25 nodos**.
    - **Condición de Vuelta (Salir de Huir):** La distancia al fantasma peligroso vuelve a ser **mayor o igual a 30 nodos** (Margen de 5 nodos para estabilizar el comportamiento), o el fantasma pasa a ser comestible.
- De "Comer píldora" a "Comer fantasma" (Y viceversa)

    - **Condición de Ida (Entrada a Cazar):** nearestGhost != null **Y** game.isGhostEdible(nearestGhost) **Y** la distancia al fantasma comestible es **menor a 40 nodos**.
    - **Condición de Vuelta (Salir de Cazar):** El tiempo de vulnerabilidad del fantasma termina (!isGhostEdible), el fantasma es devorado, o la distancia al fantasma supera los **45 nodos**.
- De "Comer píldora" a "Ir a la píldora" (Y viceversa)

    - **Condición de Ida (Peligro preventivo):** nearestGhost != null **Y** la distancia entre Pac-Man y la píldora de poder más cercana es **menor o igual al parámetro limit** (por ejemplo, 15 nodos), anticipando que necesitará el poder pronto.
    - **Condición de Vuelta (Volver al bucle normal):** La píldora de poder es consumida, o la distancia a la píldora de poder vuelve a ser **mayor a limit + 5 nodos** si Pac-Man decide cambiar de rumbo.
- De "Ir a la píldora" a "Evitar camino"

    - **Condición de Ida (Ruta interceptada):** El camino hacia la píldora de poder contiene un fantasma no comestible a una distancia **menor a 12 nodos** del objetivo.

    - **Condición de Vuelta (Ruta liberada):** El fantasma se aleja del camino de la píldora a una distancia **mayor a 17 nodos**.

    - Estado inicial

- El estado inicial es en la puerta de inicio cuando antes de dispersarse, se queda unos segundos en la carcel y ahi sale a comer pildoras.

    - Orden de transiciones
- Prioridad de transiciones en caso de que se presenten caso de que se presenten diferentes condiciones para saltar a diferentes estados, se debe ecoger la prioridad para ejecutar un unico estado.

  Si siendo presa, puede coger la pildora si y solo si no hay un fantasma a menos de 5 nodos puede tomar la pildora, si no la ignora y sigue siendo presa pero explora caminos a salvo.

  si tiene la pildora cerca, pero no hay ningun fantasma a comer, la esquiva para evitar comerla y perder el beneficio del poder.

- Clase MsPacmanInput

Prepara los datos que necesita la maquina en cada turno a partir de la clase Game

- posicion de pacman del nodo getPacmanCurrentNodeIndex()
- posiciones de los fantasmas con getGhostCurrentNodeIndex(GHOST)
- pildoras que siguen disponibles con getActivePillsIndices()
- movimientos posibles desde mi posicion getPossibleMoves(int, MOVE)
- la distancia mas corta hacia e objetivo sea una pildora normal o de poder getShortestPathDistance(int, int, MOVE)
- Clase Actions

comportamiento de pacman y como se realiza

- Si pacman esta en modo presa, marca la secuencia de nodos del nodo actual hasta la siguiente pildora getShortestPath(int, int, MOVE) y luego optiene la longitud del camino con getShortestPathDistance(int, int, MOVE), si hay un fantasma o se cruza un fantasma, devuelve NULL y recalcula el camino otra vez.

- Para huir traza el camino para el nodo mas lejano de los fantasmas con int getFarthestNodeIndexFromNodeIndex(int, int[], DM) traza la ruta con getShortestPathDistance(int, int, MOVE) y se dirige hacia alla.

- Para dirigirse a la pildora de poder mas cercana implementa getActivePowerPillsIndices() para optener el indice y luego trazamos la ruta mas

corta hacia ella getShortestPathDistance(int, int, MOVE)

- Clase Transitions

esta clase determina cuando debe cambiarse un estado a otro

Estado presa, debe cumplirse la condicion pildora comida, si es true, pasamos al estado de cazar.

EStado de cazador, debe haberse acabado el tiempo y retorna el comportamineto normal de pacman regresado al estado default.

Estado default, debe cumplirse que este en movimiento por el laberinto y pasillos de nodos, para pasar al estado de comer pildora normal

EStado comer pildora normal, debe cumplirse que existan pildoras de poder sin comer traza la distancia hacia ellos getActivePowerPillsIndices() y se dirige a ellos, ahi puede pasar al esta cazador

Entrega:
Debes darme cada clase, implementada como se menciona, si no enteindes o algo no es claro, dimelo para corregir el detalle. No supongas nada no supongas variables ni metodos, dime para aclararlos

Antes de entregar utiliza Game.java, la calse con todos los metodos correpondientes a las transiciones y acciones, debes darme las clases MspacmanInput.java, Transitions.java, Actions.java y MsPacmanInfluencia.java(vamos a implementar la maquina en este controlador trabajndo sobre MsPacmanInfluencia.java) que ya tiene implementados mapas de influencia que incrementan la efectividad y calidad de la implementacion de la maquina de estados.

MsPacman ya esta dentro del paque clase package package es.ucm.fdi.ici.c2627.practica2.grupoHLI_CSB_EJB; las demas exportan ese paquete

Correcciones:

Correcciones:

- Las clases Input, Transition y Action las creas con las especificaciones de la orden anterior ya que especificamos en el diseño de clases los metodos de cada ieza en un objeto. Una interfaz `FSM` con `getState()` y `doIt(Input in)`
- La clase constante ya te la adjunto porque no la adjunté
- Por el momento no es necesario utilizar los mapas de influencia así que en vez de utilizar esa clase, te voy a adjuntar la clase MsPacman donde deben ser codificados el FMS con las intrucciones dadas.

- Para las distancias utilizaremos DM.PATH, evitando que atraviese las paredes con la euclidiana
- Utilizaremos getSHortestPath(int, int, MOVE) para la secuencia de nodos para el camino óptimo
- La lista de candidatos esta acá getFarthestNodeIndexFromNodeIndex(int, int[], DM) y huir del fantasma peligroso observando los cruces con getJunctionIndices()
- La distancia de los fantasmas la calculamos con getGhostCurrentNodeIndex(GHOST) y con el ultimo movimiento del fantasma cercano podemos direccionar nuestro movimiento a la parte libre. nearestGhost isEdible, podemos ir hacia el, si el fantasma no es IsEdible, nos alejamos.
- PatrullarZona, estamos moviendonos y trazamos distancias con MOVE getGhostLastMoveMade(GHOST) donde podemos ver si el fantasma esta cerca o no nearestGhost != null
- El estado inicial se cambiara teniendo en cuenta que la carcel devuelve un -1, y que la norma es que pacamn puede aparecer en cualquier nodo, el estado inicial será comer pildora comun
- **Condición de Ida (Entrada a Cazar):** nearestGhost != null **Y** game.isGhostEdible(nearestGhost) **Y** la distancia al fantasma comestible es **menor a 40 nodos**.**Condición de Vuelta (Salir de Cazar):** El tiempo de vulnerabilidad del fantasma termina (!isGhostEdible), el fantasma es devorado, o la distancia al fantasma supera los **45 nodos**.
- De "Ir a la píldora" a "Evitar camino",**Condición de Ida (Ruta interceptada):** El camino hacia la píldora de poder contiene un fantasma no comestible a una distancia **menor a 12 nodos** del objetivo.**Condición de Vuelta (Ruta liberada):** El fantasma se aleja del camino de la píldora a una distancia **mayor a 17 nodos**.
- El orden establecido de prioridades de transicion será HUir, comer fantasma, evitar camino, ir a la pildora de poder y comer pildora

dom 18 de oct, cambio de diseño a estructura jerarquica de 10 estados y clase por transicion y accion.

Estado Nuevo	Transición
HuirHaciaPildoraPoder	Huir se parte en dos: correr a un cruce lejano (`HuirACruce`) o, si llegamos antes que los fantasmas, correr a la píldora de poder
EsperarJuntoPildoraPoder	Dentro de PODER: esperar junto a la píldora a que se acerquen fantasmas para que valga la pena comerla

EsquivarPildoraPoder	La regla «esquivar la píldora si no hay fantasma que comer», que estaba escondida en `ComerPildora`, pasa a ser un estado visible
LimpiarUltimas	Con pocas píldoras, recorrido corto en vez de ir a la más lejana

Estructura final (compuestos en mayúsculas):

Son 10 estados simples. Prioridad en el nivel superior: HUIR > ComerFantasma > PODER > COMER. Dentro de PODER, EvitarCamino tiene prioridad sobre IrPildoraPoder (como en el diseño).

Clases fijas de MsPacman

Clase	Que hace	Razón de ser
Constantes	Todos los umbrales con nombre y comentario	El enunciado y nuestras reglas piden un solo sitio para los números
MsPacMan	Controlador del concurso, `extends MsPacManV0`; solo cambia `getName()`	Permite elegir la mejor versión cambiando una línea
MsPacManV0	Construye las 4 máquinas, las resetea en `preCompute` y ejecuta `fsm.run`	Versión 0 = el diseño tal cual
MsPacManInput	Lo que se lee del juego en un turno: fantasma peligroso y comestible más cercanos, píldoras, distancias por pasillos, rutas	Acciones y transiciones leen de aquí; una sola clase calcula cada dato (el `Input` es lazy donde conviene)

Actions: Acciones que realiza pacman en cada estado

Clase	Estado	Que hace
MsPacManAction	Clase Abstracta	Construye el `Input`, devuelve el movimiento forzado en

		pasillos y delega en `decide()`; trae `legal()` y `movimientoSeguro()`
ComerPildoraAction	ComerPildora (inicial)	Se dirige a la pildora mas cercana(come 5)sin pasar por una de poder sin que un fantasma le corte el camino.
PatrullarAction	Patrullar	Va al cruce al que llegamos con ventaja ≥ 2 sobre todos los fantasmas, el más lejano de ellos
EsquivarPildoraPoderAction	EsquivarPildoraPoder	Movimiento seguro que no pisa píldoras de poder
LimpiarUltimasAction	LimpiarUltimas	Elige la píldora que minimiza «llegar + siguiente más cercana»
IrPildoraPoderAction	IrPildoraPoder	Camino más corto a la píldora de poder más cercana
EvitarCaminoAction	EvitarCamino	Entre los cruces con menor rodeo hasta la píldora, va al primero cuya ruta no corta ningún fantasma
EsperarJuntoPildoraPoderAction	EsperarJuntoPildoraPoder	Se mueve sin pisar la píldora y sin alejarse más de 12 nodos, maximizando la distancia a los fantasmas
HuirACruceAction	HuirACruceAction	Cruce más lejano del fantasma peligroso más cercano (`getFarthestNodeIndexFromNodeIndex` sobre `getJunctionIndices`); si la ruta está cortada, el movimiento más seguro
HuirHaciaPildoraPoderAction	HuirHaciaPildoraPoder	Va a la píldora de poder si la ruta no está cortada y

		ningún fantasma pasa a < 6 nodos
ComerFantasmaAction	ComerFantasma	Persigue al fantasma comestible más cercano por el camino más corto

Transiciones: Acciones cuando cambia de estado

Clase	Condicion y umbrales	Donde se usa
FantasmaPeligrosoCercaTransition(limite)	Clase Fantasma no comestible, fuera de la cárcel, a < `limite` por pasillos. 25 entra / 30 sigue	HUIR (nivel superior)
ComestibleCercaTransition(limite, inclusivo)	Fantasma comestible a < `limite` (o ≤ si inclusivo). 40 entra / 45 sigue	ComerFantasma
PildoraPoderCercaTransition(limite, requierePeligroso)	Píldora de poder a ≤ `limite` y, si se pide, hay fantasma peligroso. 15 / 20	PODER
RutaPoderInterceptadaTransition(limite)	Un fantasma que corta el camino a la píldora está a < `limite` de ella. 12	IrPildoraPoder → EvitarCamino
FantasmaCercaDePildoraPoderTransition(limite)	Algún fantasma a ≤ `limite` de la píldora (se usa negada). 17	EvitarCamino → IrPildoraPoder
PildoraVigiladaTransition(limite)	Fantasma peligroso a < `limite` de la píldora común más cercana. 5 / 10	ComerPildora ↔ Patrullar
PildoraPoderAlLadoTransition(limite)	Píldora de poder a ≤ `limite`, quedan comunes y no hay comestible.	ComerPildora ↔ EsquivarPildoraPoder

	10 / 15	
PocasPildorasTransition(cuantas)	Quedan < `cuantas` píldoras comunes. 20 / 25	ComerPildora ↔ LimpiarUltimas
PocosFantasmasCercaTransition(cuantos, distancia)	Menos de `cuantos` fantasmas a ≤ `distancia.` `2 y 30`	IrPildoraPoder ↔ EsperarJunto
JuntoAPildoraPoderTransition(limite, requierePeligroso)	Estamos a ≤ `limite` de la píldora de poder. 8	IrPildoraPoder → EsperarJunto
EsperaAgotadaTransition(maxTurnos)	Llevamos `maxTurnos` esperando (contador propio de la instancia, no `static`). 60	EsperarJunto → IrPildoraPoder
VentajaEnPildoraPoderTransition(ventaja)	Llegamos a la píldora con más de `ventaja` nodos de ventaja sobre el fantasma más cercano a ella, y está a ≤ 40. 5 entra / 0 sigue	HuirACruce ↔ HuirHaciaPildoraPoder
NotTransition(t)	Se cumple cuando `t` no se cumple.	Toda condición de salida (permanencia negada)
AndTransition(a, b)	Se cumplen las dos.	Salir de un estado **y** entrar en otro de menor prioridad
OrTransition(a, b)	Se cumple una de las dos	Esperar → IrPildoraPoder (fantasmas lejos **o** tiempo agotado)
EtiquetadaTransition(t, origen)	Evalúa igual que `t`, pero su `toString()` añade `[desde ORIGEN]`	Nivel superior

MsPacmanv0:
Resultado en consola:

```
Rendimiento de MsPacMan FSM grupoHLI_CSB_EJB V0 contra Ghosts practica0 en 300
partidas (media ± error típico):
 Puntos                          8230.50 ± 218.75   (σ = 3788.92, n = 300)
 Nivel alcanzado                    1.25 ± 0.04     (σ = 0.68, n = 300)
 Tiempo de partida (turnos)      2833.99 ± 60.43    (σ = 1046.72, n = 300)
```

Resultados de la primera version de pacman, todas se van a evaluar con una media de 10 partidas por version:

Resultados:

- **Puntos:** 8230,50 ± 218,75 (σ = 3788,92). La desviación es casi la mitad de la media, así que el rendimiento varía mucho de una partida a otra.
- **Nivel alcanzado:** 1,25 ± 0,04. 41 partidas (13,7 %) se quedan en el nivel 0, 144 (48,0 %) llegan al 1 y 115 (38,3 %) llegan al 2. Ninguna pasa del 2.
- **Tiempo de partida:** 2834 ± 60 turnos (σ = 1047), de un máximo de 4000..

Problemas detectas:

- Hundimiento temprano: Muere tres veces en el primer laberinto casi sin usar píldoras de poder. identificamos que puede ser la huida (`HuirACruce`) ya que corra a un cruce que se corta, o que no llegue a usar la píldora como escape porque `HuirHaciaPildoraPoder` exige llegar con más de 5 nodos de ventaja.
- Pildoras mal aprovechadas: en la clase PODER se entra con cualquier fantasma no comestible fuera de la cárcel, y solo se espera junto a la píldora si hay menos de 2 fantasmas a 30 nodos. Un fantasma a 30 nodos queda casi fuera de alcance mientras dura el tiempo de comestible, así que probablemente se come con los fantasmas lejos.

Version 1:

```
endimiento de MsPacMan FSM grupoHLI_CSB_EJB V1 contra Ghosts practica0 en 300
partidas (media ± error típico):
 Puntos                          6376.67 ± 214.25   (σ = 3710.90, n = 300)
 Nivel alcanzado                    0.92 ± 0.04     (σ = 0.73, n = 300)
 Tiempo de partida (turnos)      2312.75 ± 63.71    (σ = 1103.50, n = 300)
```

Resultados:

- La dispersión baja (σ de 4145 a 2324), pero porque desaparecen las partidas altas. V0 tenía cuatro partidas de 9000 o más y el máximo de V1 es 8940. La mediana pasa de unos 6865 a unos 5080.
- En cuanto a la primera versión no mejora mucho, de hecho por su caidas en varias partidas afecta el rendimiento de pacman, visualizando
- El hundimiento temprano no desaparece. Hay una partida por debajo de 2000 (1540) frente a dos en V0 (1470 y 1290). Sin ella la media de V1 sería unos 5460, así que cuesta unos 390 puntos de media. Tres partidas se quedan en el nivel 0 (1540, 2740 y 3260) y ninguna pasa del nivel 1. El nivel medio baja de 1,10 a 0,70.
- Todas acaban con las 3 vidas perdidas. Las partidas no terminan por el límite de 4000 turnos, sino porque Ms. Pac-Man muere. La puntuación es, por tanto, cuánto come antes de morir.

- Los puntos vienen de los fantasmas. Las partidas con 11 fantasmas dan unos 8000–9000 puntos y las de 1–2 fantasmas, menos de 3000. Con V1 hay menos píldoras de poder comidas, y de ahí que baje la media.

Version 3:

Sobre la versión del Pac-Man de Elkien me puse a trabajar yo para intentar mejorarla de manera conceptual. En sus dos versiones llegaba a unos 2100 puntos de media contra los fantasmas de Héctor, que la verdad son grandes fantasmas.

Renombré un par de cosas para que fuese mía de verdad, aunque conceptualmente es muy parecida. En vez de los superestados HUIR, COMER y PODER uso RECOLECTAR, SUPERVIVENCIA y PODER. Los comportamientos tan aislados no eran la solución, así que en RECOLECTAR junto todo lo que suma puntos con seguridad: ir a por píldoras, limpiar zonas aisladas y preparar el cebo con la píldora de poder.

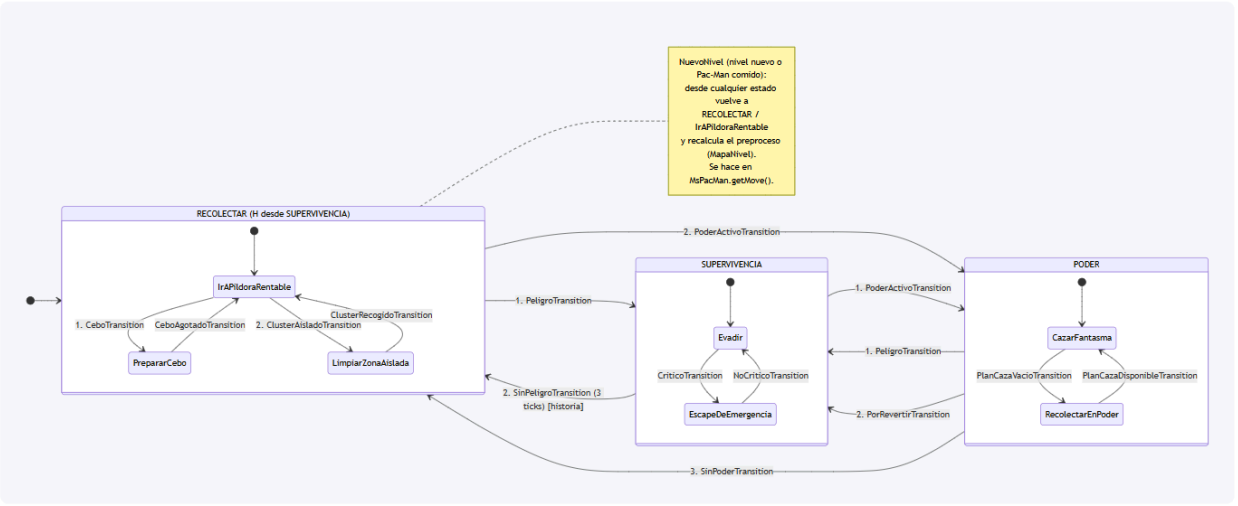
La idea de fondo también cambia. En V0 y V1 salían 1,1–1,2 fantasmas por píldora de poder, y el cuarto fantasma de una cadena vale más que los tres primeros juntos. Por eso V3 se diseña para guardar las píldoras de poder y comérselas solo cuando hay varios fantasmas a tiro.

Y para rediseñarlo puse dos cosas sobre la mesa:

Peor caso ante todo**.** Como no sabemos cómo se mueven los fantasmas, Pac-Man asume que todo fantasma peligroso va hacia él a velocidad máxima. Así la máquina no depende de adivinar patrones.

Los puntos están gordos están en encadenar**.** Las píldoras valen poco; lo que dispara la puntuación es comer 200, 400, 800 y 1600 con una sola píldora de poder. Por eso se guarda para cuando hay fantasmas agrupados cerca.

Grafo de la máquina tal como está montada en `MsPacMan.java`: 3 superestados, 7 estados hoja y 13 clases de transición.



Los números indican la prioridad: el orden de `add()`, que es el orden en que se evalúan las transiciones de un mismo estado origen.

Esta fue la idea, otra forma de verlo es así:

Superestado	Estados hoja	Para qué
RECOLECTAR (inicial, con historia)	IrAPildoraRentable (inicial), LimpiarZonaAislada, PrepararCebo	Sumar puntos sin riesgo y montar el cebo
SUPERVIVENCIA	Evadir (inicial), EscapeDeEmergencia	Salir de una amenaza; la emergencia es cuando ya no hay salida segura
PODER	CazarFantasma (inicial), RecolectarEnPoder	Encadenar fantasmas y, si no hay ninguno alcanzable, comer sin riesgo

Este diseño a parte de simplificar que nunca está demás es eficiente respecto a la implementación anterior porque no se cae en estados de negación, o estados cuya entropía podríamos decir es poca (es que últimamente me estoy matando a videos de Quantum Fracture) y tender hacia un sistema lo más libre posible.

¿Por qué creamos Navegacion, Ruta, MapaNivel y MemoriaPacMan?

Las plantillas de la práctica solo traen Input, Action, State y Transition. Al programar vimos que hacían falta cuatro clases de apoyo, cada una para resolver un problema concreto.

MapaNivel. Cada laberinto es distinto, pero dentro de un nivel no cambia. Por eso, al empezar cada nivel, calculamos una sola vez lo que es fijo: dónde están los callejones y los túneles, lo abierta que es cada zona y el tamaño del laberinto. Si lo calculásemos en cada tick, no nos daría tiempo en los 40 ms.

Ruta. En nuestro juego Pac-Man no puede darse la vuelta. Al entrar en un pasillo tiene que llegar hasta el siguiente cruce sí o sí, así que no basta con mirar la casilla de al lado. Una Ruta guarda qué pasaría si tomamos un movimiento: los nodos del pasillo, las píldoras que comeríamos, si hay una píldora de poder y cuánta ventaja le sacamos a los fantasmas.

Navegacion. Las 7 acciones hacen casi lo mismo: miran las rutas posibles y eligen la mejor, solo que cada una con un criterio distinto (recoger píldoras, huir, ir a por un fantasma...). Navegacion reúne ese código común para no repetirlo en cada acción, igual que hicimos con Movimientos en los fantasmas.

MemoriaPacMan. El Input se crea de nuevo en cada tick, así que no recuerda nada del anterior. Pero hay cosas que sí necesitamos recordar: de qué superestado venimos, para la historia de RECOLECTAR, y cuánto tiempo llevamos esperando en el cebo y cuándo lo abandonamos. Eso se guarda aquí.

Con esta aproximación se consiguieron medias de 3800 puntos contra los fantasmas de Hector y 4800 contra los fantasmas avanzados lo que la verdad es un logro, y lo lamento por no documentar más Pablo pero es que nos estamos muriendo.